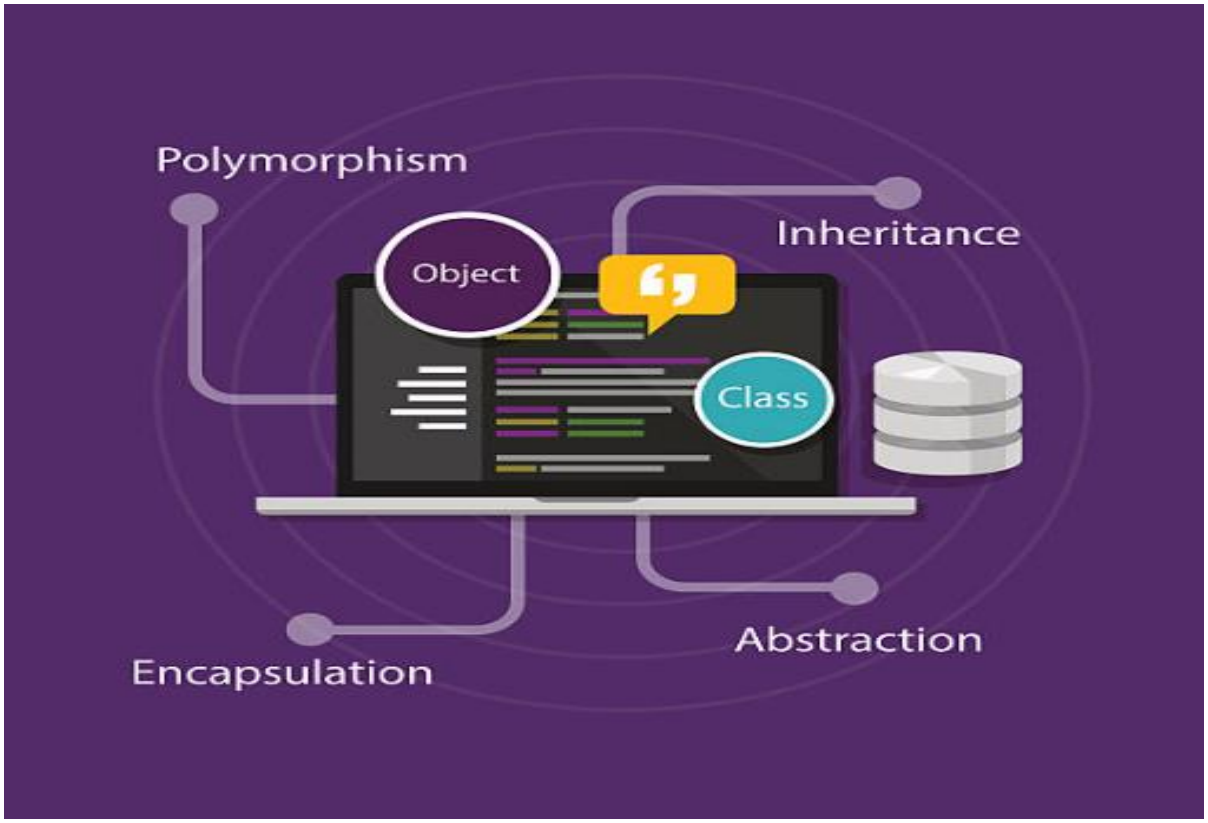
# Project Report: Railway Reservation Management System (RRMS)

## Using Object-Oriented Programming Principles




**Submitted By:**

**Mudassar SAIF (24-CS-419)**
**Kaifayat Ali (24-CS-415)**

**Submitted To:**

**Sir Faizan Iqbal**

# Contents

1. Abstraction: ..... 3
2. Introduction: ..... 3
3. System Diagrams: ..... 3
3.1. E-R Diagram ..... 3
Database Schema & ER Diagram (Table Format) ..... 4
3.2. Object Relational DIAGRAM: ..... 6
3.3. UML Class Diagram ..... 6
4. Techniques Used: ..... 8
5. Tools and Technologies Used ..... 9
6. Implementation ..... 9
6.1. Main Form ..... 9
6.2. Reservation Form ..... 9
6.3. TrainMaster Form ..... 9
6.4. PassengerMaster Form ..... 9
6.5. Cancellation Form ..... 9
6.6. Login From ..... 10
7. Conclusion: ..... 10

# Project Title:
# RRMS using Windows Forms in C# with Oracle Database

## 1. Abstraction:

The Railway Reservation Management System (RRMS) is a desktop application developed using C# Windows Forms and Oracle Database. This system simplifies the process of booking, managing, and canceling train tickets. By leveraging object-oriented programming (OOP) principles—encapsulation, inheritance, polymorphism, and abstraction—the system maintains modularity, scalability, and maintainability. Key modules include passenger management, train scheduling, reservation processing, and ticket cancellation. This report explores the design, implementation, and functionality of RRMS, offering a practical example of OOP in real-world system development.

## 2. Introduction:

Modern transportation systems demand efficient and automated solutions for ticketing and passenger management. The RRMS addresses this by replacing manual reservation processes with a robust software application. The project is structured around OOP concepts to ensure clarity and scalability. Each form in the system corresponds to a class (e.g., **MainForm**, **ReservationForm**, **CancellationForm**) that encapsulates specific functionality. The application provides user authentication, ticket booking, passenger and train management, and cancellation features.

## 3. System Diagrams:

### 3.1. E-R Diagram

**Entities:**

- Passenger
- Train
- Reservation
- Travel
- Cancellation

**Relationships:**

- A Passenger can make multiple Reservations.
- A Reservation is associated with one Train and one Travel instance.
- A Cancellation references one Reservation.

# Database Schema & ER Diagram (Table Format)

## Tables:

### 1)TrainTbl

| TrainTbl | |
|---|---|
| (PK) → | TrainId |
| | TrainName |
| | TrainCap |
| | TrainStatus |

### 2)PassengerTbl

| PassengerTbl | |
|---|---|
| (PK) → | PId |
| | PName |
| | PAdd |
| | PGender |
| | PNat |
| | PPhone |

## 3) TravelTbl

| TravelTbl | |
|---|---|
| (PK) → | TravCode |
| | TravDate |
| | Train |
| | Src |
| | Dest |
| | Cost |

## 4) ReservationTbl

| ReservastionTbl | |
|---|---|
| (PK) → | TicketId |
| | PId |
| | PName |
| | TravCode |
| | TravDate |
| | TravSrc |
| | TravDest |
| | TravCost |

## 5) CancellationTbl

| CancellationTbl | |
|---|---|
| (PK) → | CancId |
| | TicketId |
| | CancDate |

## 3.2. Object Relational DIAGRAM:

**Tables and Object Mapping**:

- Passenger ⟷ Passenger Class
- Train ⟷ TrainMaster Class
- Reservation ⟷ Reservation Class
- Travel ⟷ TravelMaster Class
- Cancellation ⟷ Cancellation Class

All classes map directly to database tables via Oracle Data Access objects using commands such as **OracleCommand**, **OracleDataAdapter**, and parameters for secure execution.

## 3.3. UML Class Diagram

| |
|---|
| **Class: Form**<br>**<< Base Class>>** |
| **Class: Main Form**<br>**<<Inherits: Form>>** |
| + MainForm()<br>+ OpenReservationForm()<br>+ OpenCancellationForm()<br>+ OpenTrainMasterForm()<br>+ OpenPassengerMasterForm()<br>+ OpenTravelMasterForm()<br>+ Logout() |

**Class: Login Form**
**<<Inherits: Form>>**

+ LoginForm()
+ button1_Click()
+ ValidateUser()
+ OpenMainForm()

**Class: TrainMasterForm**
**<<Inherits: Form>>**

+ TrainMasterForm()
+ AddTrain()
+ UpdateTrain()
+ DeleteTrain()
+ LoadTrains()
+ ResetFields()

**Class: PassengerMasterForm**
**<<Inherits: Form>>**

+ PassengerMasterForm()
+ AddPassenger()
+ UpdatePassenger()
+ DeletePassenger()
+ LoadPassengers()

**Class: TravelMasterForm**
**<<Inherits: Form>>**

+ TravelMasterForm()
+ AddTravel()
+ UpdateTravel()
+ DeleteTravel()
+ LoadTravels()

| **Class: ReservationForm**<br>**<<Inherits: Form>>** |
|---|
| + ReservationForm()<br>+ GetPassenger()<br>+ GetTravel()<br>+ MakeReservation()<br>+ LoadReservationData()<br>+ ClearFields() |

| **Class: CancellationForm**<br>**<<Inherits: Form>>** |
|---|
| + CancellationForm()<br>+ FillTicketId()<br>+ CancelTicket()<br>+ populate()<br>+ remove() |

# 4. Techniques Used:

## ➢ Object-Oriented Programming (OOP):

- **Encapsulation:** Each form encapsulates relevant functionality.
- **Inheritance:** All forms inherit from Form, the base Windows class.
- **Polymorphism:** Similar structure across button click events across forms.
- **Abstraction:** Users interact with form methods without internal implementation details.

## ➢ Oracle Database Integration:

- Parameterized queries to prevent SQL injection.
- Connection Handling using **OracleConnection**, **OracleCommand**, and **OracleDataAdapter.**

# 5. Tools and Technologies Used

| Technology | Purpose |
|---|---|
| C#(.Net Framework) | Main language for Windows Forms UI |
| Oracle Database | Backend database for persistent storage |
| Oracle.ManagedDataAccess | Oracle .NET library for DB connection |
| Windows Forms (WinForms) | GUI development |

# 6. Implementation

## 6.1. Main Form

- Act as the control center.
- Launches other forms: Reservation, Cancellation, TrainMaster, etc.
- Inherits from Form.

## 6.2. Reservation Form

- Handles ticket booking.
- Encapsulates logic for fetching passengers and travel options.
- Secure database communication via parameterized queries.

## 6.3. TrainMaster Form

- Role: Manage train data (Add/Update/Delete).
- Encapsulation of database methods.
- Uses Oracle objects for backend interaction.

## 6.4. PassengerMaster Form

- Role: Manage passenger information.
- Implements CRUD operations within form class.
- Demonstrates reuse of validation and database connection methods.

## 6.5. Cancellation Form

- Fetches and removes reservations.
- Uses abstraction to hide DB complexity from UI interactions.

## 6.6. Login From

- Authenticates users.
- Inherits from Form, encapsulates login validation logic.

# 7. Conclusion:

The Railway Reservation Management System is a practical application of object-oriented programming in real-world software design. The project follows OOP principles throughout all forms and modules, leading to a clean, maintainable, and scalable system. The addition of new modules like ticket rescheduling shows the flexibility of the object-oriented design. Integration with Oracle ensures robust data management, while Windows Forms provides a user-friendly interface. Overall, this project demonstrates how modular design and OOP principles enhance software quality and efficiency.